

C++&Qt project 레포트

로봇 예비단원 주호진

1. 과제

개인별로 C++, QT를 사용한 프로그램 제작

필수:

- ros-qt 패키지 사용
- 패키지 1개에 노드 2개 이상
- 노드끼리 topic 주고받기
- 노드끼리 service/action중 택1 주고받기
- 9월 14일 월요일 오후 9시반 발표예정(ppt 포함)

2. 구현

주제: 경찰과 도둑 게임(알고리즘을 강화한)

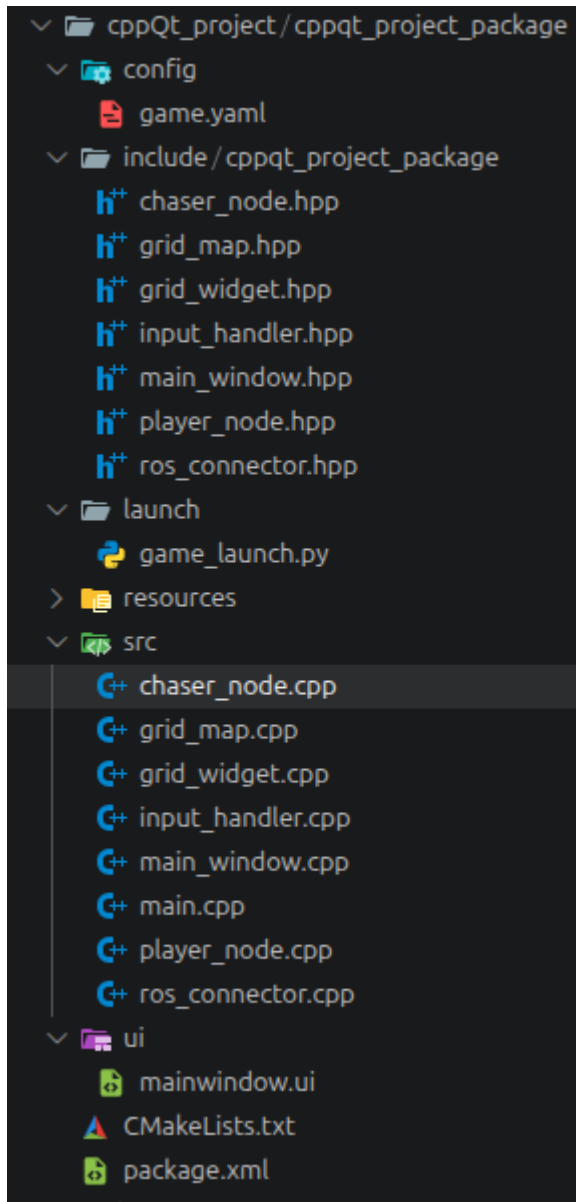
아오오니라는 게임의 느낌을 구현하고자 했다.

아오오니는 오니가 나왔을 때 확 느껴지는 특징이 닿을 듯 말 듯한 텐션이 특징이다.

chaser(오니)가 플레이어보다 속도가 빠르고, 이 속도 갭은 맵이동이나 장애물 등을 이용해서 따돌려야 하는 이 텐션을 구현하고자 했다. 해당 게임에서는 속도 갭을 극복하는 수단을 방향 회전에 두기로 했다.

따라서, 기존에 이동하던 방향이 아닌 경우 속도 지연이 생기게 만들었다.

- 폴더 구조



Chaser: 쫓는 사람

Grid map: 격자를 생성

Grid widget: 격자를 그림

Input handler: 키입력

Player: 쫓기는 사람(플레이어)

Ros connector: qt의 통신을 분리한 파일

- chaser

chaser_node.h

```
chaser_node:
  ros_parameters:
    grid_size: 20
    chaser_update_interval_ms: 155
    chaser_start_x: 18
    chaser_start_y: 18
```

격자를 사용해 Update interval이 짧을수록 이동속도가 빠른 것처럼 구현하게 되었다.

```
chaser_node::chaser_node()
// BFS
std::vector<std::vector<bool>> visited(grid_size_, std::vector<bool>(grid_size_));
// 경로 저장 parent
std::vector<std::vector<std::pair<int, int>>> parent(grid_size_, std::vector<std::pair<int, int>>(grid_size_));
std::queue<std::pair<int, int>> q;

q.push({chaser_x_, chaser_y_});
visited[chaser_y_][chaser_x_] = true;

int dx[] = {-1, 0, 1, 0};
int dy[] = {0, 1, 0, -1};
bool found = false;

// 큐 빌 때까지
while (!q.empty())
{
    auto [cx, cy] = q.front();
    q.pop();

    if (cx == target_x && cy == target_y) // 도달
    {
        found = true;
        break;
    }

    for (int i = 0; i < 4; i++) // 맨해튼 거리이므로 4방향
    {
        int nx = cx + dx[i];
        int ny = cy + dy[i];

        // 방문 가능(벽x)하면서 방문하지 않은 경우
        if (grid_map_.isWalkable(nx, ny) && !visited[ny][nx])
        {
            visited[ny][nx] = true;
            parent[ny][nx] = {cx, cy};
            q.push({nx, ny});
        }
    }
}
```

일단 격자이기 때문에 가중치가 동일하다. 동일한 가중치에서 좋은 성능을 뽑아내는 bfs를 chaser의 알고리즘으로 채택하게 되었다.

```
int next_dx = curr_x - chaser_x_;
int next_dy = curr_y - chaser_y_;
```

```
// 방향이 다르면
if ((next_dx != last_dx_ || next_dy != last_dy_)
{
    if (!is_turning_delay_)
    // 회전 딜레이
    {
        is_turning_delay_ = true;
        return;
    }
}
```

좌표를 비교해서 방향이 다른 경우 delay 변수가 켜지면서 1회 루프를 더 돌면서 지연이 생긴다.

```
// 같으면 그냥 진행
is_turning_delay_ = false;
last_dx_ = next_dx;
last_dy_ = next_dy;
chaser_x_ = curr_x;
chaser_y_ = curr_y;
```

지연이 생긴 뒤에는 if문을 넘어가고 그 아래에서 다시 turning delay를 false로 변경해준다.

```
chaser_pub_ = this->create_publisher<geometry_msgs::msg::Point>("chaser_position", 10);

player_pos_sub_ = this->create_subscription<geometry_msgs::msg::Point>(
    "player_position", 10, std::bind(&ChaserNode::playerPosCallback, this, std::placeholders::_1));

reset_srv_ = this->create_service<std_srvs::srv::Empty>(
    "reset_chaser", std::bind(&ChaserNode::resetCallback, this, std::placeholders::_1, std::placeholders::_2));
```

Chaser는 player의 좌표를 구독하고, qt에서 게임이 재시작될 때 초기화된 좌표를 퍼블리쉬한다.

- player

```
player_pub_ = this->create_publisher<geometry_msgs::msg::Point>("player_position", 10);
stamina_pub_ = this->create_publisher<std_msgs::msg::Float32>("player_stamina", 10);

command_sub_ = this->create_subscription<geometry_msgs::msg::Point>(
    "player_command", 10, std::bind(&PlayerNode::commandCallback, this, std::placeholders::_1));

reset_srv_ = this->create_service<std_srvs::srv::Empty>(
    "reset_player", std::bind(&PlayerNode::resetCallback, this, std::placeholders::_1, std::placeholders::_2));
game_over_srv_ = this->create_service<std_srvs::srv::Empty>(
    "game_over_player", std::bind(&PlayerNode::gameOverCallback, this, std::placeholders::_1,
    std::placeholders::_2)); // 1:요청, 2: 응답
```

플레이어는 gui 상에서 받아온 키입력을 구독하고, game over시 응답을 받아 멈추고, reset시 초기화된 좌표를 퍼블리쉬한다. 만약 정지하지 않았다면 client가 알게 된다.

```

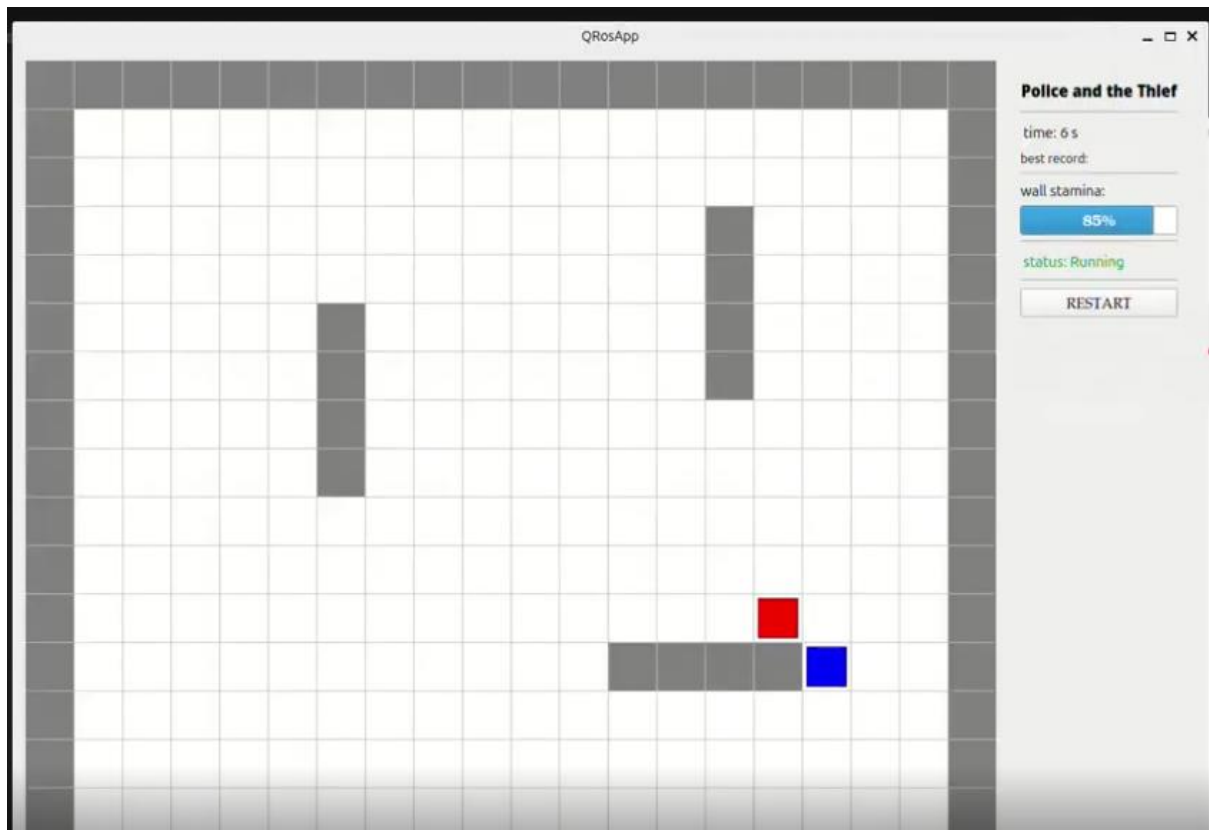
if (stamina_exhausted_) // 고갈
{
    exhaust_counter++;
    if (exhaust_counter_ > 10) // 10 x 50 = 500ms
    {
        stamina_exhausted_ = false;
        exhaust_counter_ = 0;
    }
}
else
{
    // 벽에 붙어있다면 -, 아니면 +
    if (isTouchingWall())
    {
        wall_stamina_ -= consumption_rate_;
        if (wall_stamina_ <= 0.0)
        {
            wall_stamina_ = 0.0;
            stamina_exhausted_ = true; // 고갈
        }
    }
    else
    {
        wall_stamina_ += recovery_rate_;
        if (wall_stamina_ > max_stamina_)
            wall_stamina_ = max_stamina_;
    }
}
}

```

플레이어에게 방향 변경이라는 무기를 쥐어준 대가로, 방향 변경을 쉽게할 수 있는 벽같은 구간 곁에 있을 시에 스테미너를 설정하여 닳게 만들었다. 해당 스테미너가 전부 소진되면 스텐에 걸리게 된다.

3. 작동 영상

[작동 영상](#)



벽에 맞닿아있으면 스테미너가 줄어들게 되고, 다 줄어들면 스텐에 걸리게 됨(사실상 실패)

4. AI 사용 보고서

<https://share.gemini.google/s7jDjBXyOJ6x>

속도는 ai가 더 빠르고 코너링은 느리게 해서 플레이어가 장애물을 이용해서 버티도록 설계하고 싶은데 기본 틀이랑 시작을 어떻게 해야할까

아오오니는 무슨 알고리즘으로 찾아오게 하는게 좋을까

핵심 알고리즘: BFS와 거리 필드

- 플레이어의 현재 타일을 기준으로 맵 전체의 모든 타일까지의 최단 거리(Step Count)를 매 프레임(또는 일정 주기마다) 계산합니다.

아이디어를 적용할 수 있는 기법 등을 질문했다. 원하는 조건 등을 입력하고 어떤 기법을 사용할지 채택했다. 처음에는 격자 형태로 만들어서 격자를 늘려서 촘촘히 만든 뒤 그래픽을 씌워서 격자가 없는 것처럼 구현하려고 했었다.

해당 게임에서 node service나 action 통신 기능을 넣어야 한다면 어디에 넣을 수 있을까

서비스나 액션을 어느 부분에서 적용할 수 있을지 물어봤다. 게임 오버, 재시작에서 서비스를 쓰는 게 관례라고 답변을 받았는데 사실 예시를 듣고 해당 상황에 서비스를 쓰는 게 좋다는 직관이 오지 않아 추가 질문을 했다.

서비스랑 토픽 차이점?

서비스도 받으면 콜백이 나가는 건 비슷해보이는데

두 방식 모두 데이터를 받을 때 콜백 함수가 실행되기 때문에 겉보기엔 비슷해 보이지만, **데이터 흐름의 방향성과 지속성**에서 결정적인 차이가 있습니다. 토픽은 끊임없이 흐르는 '방송(Broadcast)'이고, 서비스는 명확한 '문답(Request-Response)'입니다.

토픽으로 구현해도 기능이 동일해보여서 비슷한 질문을 여러개 했다.

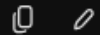
리스폰스를 활용하지 않는다는 가정하에 토픽이랑 거의 동일한 건가

아니요, 응답을 무시하거나 비워둔다고 가정해도 토픽과 서비스는 구조와 동작 방식에서 근본적인 차이가 있습니다.

리스폰스에 아무 값이 없으면 토픽과 다른 점이 무엇인지 물어보았다. 아래도 거의 비슷한 질문을 했는데, 답변은 사실 다 비슷하긴 했다. 그냥 이해가 될때까지 비슷한 질문을 했다. 토픽은 보내고 상대가 수신했는지에 대해 아예 상관을 하지 않으므로 이러한 점이 아예 다른 부분이다.

```
(void)req;  
(void)res;
```

근데 이렇게 두면 사용 안하는거 아니야?



맞습니다. 로직상으로는 **실제로 사용하지 않는 것**이 맞습니다.

그럼에도 저렇게 작성하는 이유는 컴파일러의 "**사용되지 않는 변수 경고(Unused Parameter Warning)**"를 방지하기 위한 표준 C/C++ 관용구이기 때문입니다.

받았는지 알수있는 사실만으로 서비스와 토픽이 다르다



네, 맞습니다. "상대방이 내 요청을 받았고, 처리를 완료했으며, 그 결과(또는 완료 신호)를 돌려받았다"는 응답 보장성(Acknowledgment)이 토픽과 서비스를 구분하는 가장 결정적인 차이입니다.

위아래로 와리가리 치면 무한 반복하는데 이거 막으려면 어떻게 해



플레이어가 위아래로 왕복할 때 AI가 같은 칸 사이를 진동하며 무한 반복하는 현상은 **거리 값이 같은 인접 타일(동점 상황)에서 선택 기준이 매번 흔들리거나, AI에게 이전 이동 방향에 대한 기억(관성)이 없을 때** 발생합니다.

처음에 chaser와 플레이어의 속도 차이가 거의 없었는데 chaser에게 방향을 꺾을 때 패널티를 주니 플레이어가 위아래로 진동하면서 움직일 경우 해당 패널티가 두 속도차를 거의 비슷하게 매꾸면서 플레이어 기록에 유리한 무적존이 생기게 되어서 질문했다.

이에 대한 다양한 답변을 받았는데 결과부터 말하면 위아래 진동 방지 알고리즘을 적용하진 않았고, 구현하면서 가장 중요하게 생각한 난이도 문제도 같이 있었어서 해당 문제를 위해 속도를 조절하다보니까 bfs의 진동 문제가 없어지진 않았지만 결과적으로 무적존이 사라지게 되었다.

결론적으로는 속도를 올려서 무적존이 사라지게 되는 방법을 임시 방편으로 채택하게 됐다.